

Bookshare: Abstract

Eine Buchtausch-App muss Bücher, Nutzer und Ausleihen miteinander verknüpfen. In diesem Projekt wird das mithilfe einer relationalen Datenbank gelöst. Nutzer bieten Bücher aus dem eigenen Bestand zum Verleihen an, fragen Bücher anderer Nutzer an und bilden den Ausleihvorgang über die Anwendung ab.

Inhaltlich besteht die Lösung aus vier Modulen, die zwar fachlich getrennt sind, aber im Schema über Fremdschlüssel zusammenarbeiten.

- Der Katalog enthält die Metadaten eines Buches in der dritten Normalform. Das physische Exemplar des Buches bildet die Schnittstelle zwischen Katalog und Ausleihprozess. Die Trennung von Metadaten und Exemplar erlaubt, dass mehrere Nutzer das gleiche Buch in unterschiedlichem Zustand anbieten, ohne die Buchdaten redundant pflegen zu müssen.
- Der Ausleihprozess ist als Abfolge eigener Tabellen für Anfrage, Ausleihe und Rückgabe modelliert, weil jeder Schritt eigene Daten trägt, die jeweils erst durch entscheidende Ereignisse erzeugt werden.
- Die Benutzerverwaltung führt Profile, Adressen und Rollen getrennt. Einem Nutzer können mehrere Rollen und mehrere Adressen zugeordnet werden, ohne dass seine Stammdaten doppelt gehalten werden müssen. Eine Adresse ist im Kontext einer Ausleihe ein Bewegungsdatum.
- Bewertungen setzen eine abgeschlossene Ausleihe voraus und richten sich an das Buch oder den anderen Nutzer.

Zum Start der Entwicklung wurden Umgebung und Werkzeuge gewählt sowie ein Repository auf Github eingerichtet.

Danach wurden Rollen bestimmt und Usecases identifiziert. Aus den Rollen und ihren Handlungen wurden die Usecases abgeleitet und funktional in Epics, Stories und Tasks zerlegt. Jedem Task wurden die betroffenen DB-Tabellen zugeordnet. Damit stand früh im Projekt fest, welche Tabelle welche Funktionen unterstützen muss. Die funktionale Zerlegung liegt im Repository als PDF-Dokument im Verzeichnis „dokumentation“.

Aus der Zerlegung entstanden Datenwörterbuch und Entitäten. Das Datenwörterbuch hält je Attribut Typ und Bedeutung fest. Darauf folgte das ER-Modell mit Entitäten, Beziehungen und Kardinalitäten. Das Modell wurde schließlich in SQL-Statements übersetzt. Das `create_tables` Skript droppt alle Objekte vorab und legt das Schema bei jedem Lauf neu an, sodass es sich während der Entwicklung schnell neu aufsetzen ließ.

Die Buch- und Autorendaten wurden über die Open Library API bezogen, per Python-Skript aufbereitet und um Herkunftsland und Beschreibung der Autoren aus Wikidata ergänzt. Über die APIs ließen sich größere und plausiblere Datenmengen gewinnen als von Hand, sodass sich die Abfragen an einem belastbaren Bestand testen ließen, etwa an Büchern mit mehreren Autoren.

Am Ende entstand für jeden Usecase eine Beispielabfrage, darunter die räumliche Suche per Haversine-Formel, überfällige Ausleihen und ungelesene Nachrichten. Zusätzlich wurde die PoC-Anwendung in Python auf Basis von Flask gebaut. Es spielt den Ausleihvorgang von der Anfrage bis zur Bewertung durch und greift direkt auf die Oracle-Datenbank zu. Damit war geprüft, dass das Schema die Prozesse im Zusammenspiel trägt.

Insgesamt setzt sich der Entwurf des Datenbankschemas aus mehreren Dutzend Einzelentscheidungen zusammen, die sich unterschiedlichen Kategorien zuordnen lassen. Sie sind im Abschnitt Entwurfsentscheidungen der README des GitHub-Repositorys dokumentiert.

In einer produktiven Umsetzung ließe sich die Datenhaltung je Modul auf eigenständige Dienste aufteilen. Die Authentifizierung würde an einen Identity-Provider ausgelagert und die zugehörigen Felder im Schema entfernt. Naheliegend wäre außerdem eine Tabelle für Streitfälle zwischen Leiher und Verleiher.